

SAP BusinessObjects 4.2 SP5 Patch 3

Local CMS Connection Test and Scheduled Report / Publication Inventory Extractor

This guide contains the complete test program and the initial read-only extraction program discussed in the response. The utilities are intended to run on a local Windows PC using the SAP BusinessObjects Java SDK JAR files and connect remotely to the CMS.

1. Local folder structure

```
C:\BOInventory\  
  lib\  
  src\  
  output\  
  config.properties  
  compile.bat  
  run-test.bat  
  run-extract.bat
```

Place the SAP BO 4.2 SP5 Patch 3 Java SDK JAR files in the **lib** directory. Use a Java version compatible with the SDK installed with the BO 4.2 environment.

config.properties

```
cms=YOUR-CMS-SERVER:6400  
username=YOUR_USERNAME  
authentication=secEnterprise  
output=output/bo_schedule_inventory.csv
```

The password should be requested at runtime rather than stored in this file.

2. CMS connection test

Create **src\BOConnectionTest.java**. This program logs in, obtains InfoStore, runs a small CI_INFOOBJECTS query, prints returned objects, and logs off.

```
import java.io.Console;  
import java.util.Scanner;  
  
import com.crystaldecisions.sdk.framework.CrystalEnterprise;  
import com.crystaldecisions.sdk.framework.IEnterpriseSession;  
import com.crystaldecisions.sdk.occa.infostore.IInfoObject;  
import com.crystaldecisions.sdk.occa.infostore.IInfoObjects;  
import com.crystaldecisions.sdk.occa.infostore.IInfoStore;  
  
public class BOConnectionTest {  
  
    public static void main(String[] args) {  
  
        if (args.length < 3) {  
            System.out.println(  
                "Usage: BOConnectionTest <CMS:PORT> <USERNAME> <AUTHENTICATION>"  
            );  
            System.out.println(  
                "Example: BOConnectionTest boserver:6400 Administrator secEnterprise"  
            );  
            System.exit(1);  
        }  
  
        String cms = args[0];  
        String username = args[1];  
        String authentication = args[2];  
  
        String password = readPassword();  
        IEnterpriseSession session = null;  
  
        try {  
            System.out.println();  
            System.out.println("=====");  
            System.out.println(" SAP BusinessObjects SDK Connection Test");  
            System.out.println("=====");  
            System.out.println("CMS          : " + cms);  
            System.out.println("Username      : " + username);  
            System.out.println("Authentication : " + authentication);  
            System.out.println();  
            System.out.println("Connecting...");  
  
            session = CrystalEnterprise.  
                .getSessionMgr()
```

```

        .login(username, password, cms, authentication);

System.out.println("LOGIN SUCCESSFUL");
System.out.println("CMS connected.");

IInfoStore infoStore =
    (IInfoStore) session.getService("InfoStore");

System.out.println();
System.out.println("Running test CMS query...");

String query =
    "SELECT TOP 20 " +
    "SI_ID, SI_CUID, SI_NAME, SI_KIND, SI_PARENTID " +
    "FROM CI_INFOOBJECTS " +
    "WHERE SI_INSTANCE=0 " +
    "ORDER BY SI_NAME";

IInfoObjects objects = infoStore.query(query);

System.out.println();
System.out.println("Objects returned: " + objects.size());
System.out.println("-----");

for (int i = 0; i < objects.size(); i++) {
    IInfoObject obj = (IInfoObject) objects.get(i);

    System.out.println(
        "ID=" + obj.getID()
        + " | KIND=" + obj.getKind()
        + " | NAME=" + obj.getTitle()
        + " | CUID=" + obj.getCUID()
    );
}

System.out.println("-----");
System.out.println();
System.out.println(
    "TEST SUCCESSFUL - SDK and CMS connection are working."
);

} catch (Exception e) {
    System.err.println();
    System.err.println("TEST FAILED");
    System.err.println("-----");
    System.err.println(e.getClass().getName());
    System.err.println(e.getMessage());
    System.err.println("-----");
    e.printStackTrace();
} finally {
    if (session != null) {
        try {
            session.logout();
            System.out.println("Logged off.");
        } catch (Exception ignored) {
        }
    }
}
}

private static String readPassword() {
    Console console = System.console();

    if (console != null) {
        char[] chars = console.readPassword("Password: ");
        return new String(chars);
    }

    System.out.print("Password: ");
    Scanner scanner = new Scanner(System.in);
    return scanner.nextLine();
}
}

```

3. Compile and run the connection test

```
javac -cp "lib/*" -d bin src\BOConnectionTest.java
```

```
java -cp "bin;lib/*" BOConnectionTest YOURCMS:6400 youruser secEnterprise
```

Example:

```
java -cp "bin;lib/*" BOConnectionTest BOPROD01:6400 Administrator secEnterprise
```

A successful run should report LOGIN SUCCESSFUL, return a small list of CMS objects, and then log off. If it fails with **NoClassDefFoundError**, record the exact missing class name because it identifies a missing SDK dependency.

4. Scheduled report and publication inventory extractor

Create `src\BOScheduleInventory.java`. This first full version is deliberately defensive and retains raw schedule/destination information. That diagnostic output can then be mapped to clean FTP, email, filesystem and scheduling columns for the exact SP5 Patch 3 environment.

```
import java.io.*;
import java.text.SimpleDateFormat;
import java.util.*;

import com.crystaldecisions.sdk.framework.CrystalEnterprise;
import com.crystaldecisions.sdk.framework.IEnterpriseSession;
import com.crystaldecisions.sdk.occa.infostore.*;
import com.crystaldecisions.sdk.properties.IProperties;
import com.crystaldecisions.sdk.properties.IProperty;
import com.businessobjects.sdk.plugin.desktop.publication.IPublication;

public class BOScheduleInventory {

    private static IInfoStore infoStore;

    private static final Map<Integer, String> folderCache =
        new HashMap<Integer, String>();

    private static final Map<Integer, String> principalCache =
        new HashMap<Integer, String>();

    private static final SimpleDateFormat DATE_FORMAT =
        new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");

    public static void main(String[] args) {

        if (args.length < 4) {
            System.out.println(
                "Usage: BOScheduleInventory " +
                "<CMS:PORT> <USERNAME> <AUTH> <OUTPUT.CSV>"
            );

            System.out.println(
                "Example: BOScheduleInventory " +
                "BOPROD01:6400 Administrator secEnterprise " +
                "output\\bo_inventory.csv"
            );

            System.exit(1);
        }

        String cms = args[0];
        String username = args[1];
        String authentication = args[2];
        String outputFile = args[3];

        String password = readPassword();

        IEnterpriseSession session = null;
        PrintWriter writer = null;

        try {
            System.out.println();
            System.out.println("=====");
            System.out.println(" SAP BO Scheduled Report Inventory");
            System.out.println("=====");
            System.out.println("CMS      : " + cms);
            System.out.println("User      : " + username);
            System.out.println("Output    : " + outputFile);
            System.out.println();

            System.out.println("Connecting to CMS...");

            session = CrystalEnterprise
                .getSessionMgr()
                .logon(username, password, cms, authentication);

            System.out.println("Connected.");

            infoStore =
                (IInfoStore) session.getService("InfoStore");

            File file = new File(outputFile);
            File parent = file.getParentFile();

            if (parent != null) {
                parent.mkdirs();
            }

            writer = new PrintWriter(
                new OutputStreamWriter(
```

```

        new FileOutputStream(file),
        "UTF-8"
    );

    writeHeader(writer);
    extractScheduledObjects(writer);

    System.out.println();
    System.out.println("=====");
    System.out.println("Extraction completed.");
    System.out.println("File: " + file.getAbsolutePath());
    System.out.println("=====");

} catch (Exception e) {
    System.err.println();
    System.err.println("EXTRACTION FAILED");
    System.err.println(e.getMessage());
    e.printStackTrace();

} finally {
    if (writer != null) {
        writer.flush();
        writer.close();
    }

    if (session != null) {
        try {
            session.logout();
        } catch (Exception ignored) {
        }
    }
}

}

private static void extractScheduledObjects(
    PrintWriter writer) throws Exception {

    String query =
        "SELECT " +
        "SI_ID, " +
        "SI_CUID, " +
        "SI_NAME, " +
        "SI_KIND, " +
        "SI_PROGID, " +
        "SI_PARENTID, " +
        "SI_OWNER, " +
        "SI_SCHEDULEINFO, " +
        "SI_PUBLICATION_DOCUMENTS, " +
        "SI_PRINCIPALS, " +
        "SI_PROCESSINFO " +
        "FROM CI_INFOOBJECTS " +
        "WHERE SI_INSTANCE=0";

    System.out.println("Querying CMS repository...");

    IInfoObjects objects = infoStore.query(query);

    System.out.println(
        "Candidate objects returned: " + objects.size()
    );

    int exported = 0;

    for (int i = 0; i < objects.size(); i++) {

        IInfoObject object =
            (IInfoObject) objects.get(i);

        try {
            ISchedulingInfo schedulingInfo =
                object.getSchedulingInfo();

            if (schedulingInfo == null) {
                continue;
            }

            String scheduleRaw =
                safePropertyDump(
                    object.properties(),
                    "SI_SCHEDULEINFO"
                );

            IDestinations destinations =
                schedulingInfo.getDestinations();

            boolean hasDestinations =
                destinations != null &&
                destinations.size() > 0;

```

```

String sourceDocuments = "";
String sourceDocumentFolders = "";
String recipients = "";

if (object instanceof IPublication) {
    IPublication publication =
        (IPublication) object;

    sourceDocuments =
        getPublicationDocuments(publication);

    sourceDocumentFolders =
        getPublicationDocumentFolders(publication);

    recipients =
        getPublicationRecipients(publication);
}

String folderPath =
    getFolderPath(object.getParentID());

if (hasDestinations) {
    for (int d = 0;
        d < destinations.size();
        d++) {

        IDestination destination =
            (IDestination) destinations.get(d);

        String destinationName =
            safe(destination.getName());

        String destinationProperties =
            dumpProperties(
                destination.properties(),
                ""
            );

        writeRow(
            writer, object, folderPath,
            sourceDocuments, sourceDocumentFolders,
            recipients, destinationName,
            destinationProperties, scheduleRaw
        );

        exported++;
    }
} else {
    writeRow(
        writer, object, folderPath,
        sourceDocuments, sourceDocumentFolders,
        recipients, "", "", scheduleRaw
    );

    exported++;
}

if (exported % 100 == 0) {
    System.out.println(
        "Rows exported: " + exported
    );
}

} catch (Exception objectError) {

    System.err.println(
        "Warning: unable to inspect object "
        + object.getID()
        + " / "
        + object.getTitle()
        + ": "
        + objectError.getMessage()
    );
}

}

System.out.println(
    "Total CSV rows exported: " + exported
);
}

private static String getPublicationDocuments(
    IPublication publication) {

    StringBuilder result = new StringBuilder();

```

```

try {
    Collection docs =
        publication.getSchedulableDocuments();

    Iterator iterator = docs.iterator();

    while (iterator.hasNext()) {

        Object item = iterator.next();

        if (!(item instanceof IInfoObject)) {
            continue;
        }

        IInfoObject document =
            (IInfoObject) item;

        append(
            result,
            document.getTitle()
            + " [ID="
            + document.getID()
            + ", CUID="
            + document.getCUID()
            + "]"
        );
    }

} catch (Exception e) {
    return "ERROR: " + e.getMessage();
}

return result.toString();
}

private static String getPublicationDocumentFolders(
    IPublication publication) {

    StringBuilder result = new StringBuilder();

    try {
        Collection docs =
            publication.getSchedulableDocuments();

        Iterator iterator = docs.iterator();

        while (iterator.hasNext()) {

            Object item = iterator.next();

            if (!(item instanceof IInfoObject)) {
                continue;
            }

            IInfoObject document =
                (IInfoObject) item;

            String path =
                getFolderPath(document.getParentID());

            append(
                result,
                document.getTitle()
                + " => "
                + path
            );
        }

    } catch (Exception e) {
        return "ERROR: " + e.getMessage();
    }

    return result.toString();
}

private static String getPublicationRecipients(
    IPublication publication) {

    StringBuilder result = new StringBuilder();

    try {
        Collection principals =
            publication.getPrincipals();

        Iterator iterator =
            principals.iterator();

        while (iterator.hasNext()) {

```

```

        Object value = iterator.next();

        int principalID;

        if (value instanceof Integer) {
            principalID =
                ((Integer) value).intValue();
        } else {
            principalID =
                Integer.parseInt(value.toString());
        }

        String principal =
            getPrincipalName(principalID);

        append(result, principal);
    }

} catch (Exception e) {
    return "ERROR: " + e.getMessage();
}

return result.toString();
}

private static String getPrincipalName(
    int principalID) {

    if (principalCache.containsKey(principalID)) {
        return principalCache.get(principalID);
    }

    String value = "ID=" + principalID;

    try {
        String query =
            "SELECT SI_ID, SI_NAME, SI_KIND " +
            "FROM CI_SYSTEMOBJECTS " +
            "WHERE SI_ID=" + principalID;

        IInfoObjects objects =
            infoStore.query(query);

        if (objects.size() > 0) {

            IInfoObject principal =
                (IInfoObject) objects.get(0);

            value =
                principal.getTitle()
                + " ["
                + principal.getKind()
                + ", ID="
                + principalID
                + "]";

        }

    } catch (Exception ignored) {
    }

    principalCache.put(principalID, value);

    return value;
}

private static String getFolderPath(
    int parentID) {

    if (parentID <= 0) {
        return "/";
    }

    if (folderCache.containsKey(parentID)) {
        return folderCache.get(parentID);
    }

    List<String> names =
        new ArrayList<String>();

    int currentID = parentID;
    int safety = 0;

    try {
        while (currentID > 0 && safety < 100) {

            safety++;

            String query =
                "SELECT SI_ID, SI_NAME, SI_PARENTID " +

```

```

        "FROM CI_INFOOBJECTS " +
        "WHERE SI_ID=" + currentID;

        IInfoObjects folders =
            infoStore.query(query);

        if (folders.size() == 0) {
            break;
        }

        IInfoObject folder =
            (IInfoObject) folders.get(0);

        String name = folder.getTitle();

        if (name != null &&
            name.trim().length() > 0) {
            names.add(name);
        }

        int nextID = folder.getParentID();

        if (nextID == currentID) {
            break;
        }

        currentID = nextID;
    }
} catch (Exception e) {
    return "[Folder lookup error: "
        + e.getMessage()
        + "]\n";
}

Collections.reverse(names);

StringBuilder path =
    new StringBuilder();

for (String name : names) {
    path.append("/");
    path.append(name);
}

String result =
    path.length() == 0
    ? "/"
    : path.toString();

folderCache.put(parentID, result);

return result;
}

private static String safePropertyDump(
    IProperties properties,
    String propertyName) {
    try {
        IProperty property =
            properties.getProperty(propertyName);

        if (property == null) {
            return "";
        }

        Object value = property.getValue();

        if (value instanceof IProperties) {
            return dumpProperties(
                (IProperties) value,
                ""
            );
        }

        return value == null ? "" : value.toString();
    } catch (Exception e) {
        return "ERROR: " + e.getMessage();
    }
}

private static String dumpProperties(
    IProperties properties,
    String prefix) {
    if (properties == null) {
        return "";
    }

```

```

    }

    StringBuilder result =
        new StringBuilder();

    try {
        Integer[] ids =
            properties.getPropertyIDs();

        if (ids == null) {
            return "";
        }

        for (int i = 0; i < ids.length; i++) {

            IProperty property =
                properties.getProperty(ids[i]);

            if (property == null) {
                continue;
            }

            String key = prefix + ids[i];

            if (key.toUpperCase().contains("PASSWORD")) {
                continue;
            }

            Object value = property.getValue();

            if (value instanceof IProperties) {

                String nested =
                    dumpProperties(
                        (IProperties) value,
                        key + "."
                    );

                if (nested.length() > 0) {
                    append(result, nested);
                }

            } else {

                String text =
                    value == null
                    ? ""
                    : value.toString();

                append(
                    result,
                    key + "=" + text
                );

            }
        }

    } catch (Exception e) {

        append(
            result,
            "PROPERTY_DUMP_ERROR="
            + e.getMessage()
        );

    }

    return result.toString();
}

private static void writeHeader(
    PrintWriter writer) {

    String[] columns = {
        "SI_ID",
        "CUID",
        "Object_Type",
        "Object_Name",
        "Folder_Path",
        "Owner",
        "Publication_Source_Documents",
        "Source_Document_Folders",
        "Publication_Recipients",
        "Destination_Type",
        "Destination_Properties",
        "Schedule_Properties"
    };

    writer.println(toCsv(columns));
}

```

```

private static void writeRow(
    PrintWriter writer,
    IInfoObject object,
    String folderPath,
    String sourceDocuments,
    String sourceDocumentFolders,
    String recipients,
    String destinationName,
    String destinationProperties,
    String scheduleRaw) {

    String owner = "";

    try {
        owner = object.getOwner();
    } catch (Exception ignored) {
    }

    String[] columns = {
        String.valueOf(object.getID()),
        safe(object.getCUID()),
        safe(object.getKind()),
        safe(object.getTitle()),
        safe(folderPath),
        safe(owner),
        safe(sourceDocuments),
        safe(sourceDocumentFolders),
        safe(recipients),
        safe(destinationName),
        safe(destinationProperties),
        safe(scheduleRaw)
    };

    writer.println(toCsv(columns));
}

private static String toCsv(
    String[] values) {

    StringBuilder line =
        new StringBuilder();

    for (int i = 0; i < values.length; i++) {

        if (i > 0) {
            line.append(",");
        }

        String value =
            values[i] == null
            ? ""
            : values[i];

        value = value.replace("\"", "\\\"");

        line.append("\"");
        line.append(value);
        line.append("\"");
    }

    return line.toString();
}

private static void append(
    StringBuilder builder,
    String value) {

    if (value == null ||
        value.length() == 0) {
        return;
    }

    if (builder.length() > 0) {
        builder.append(" | ");
    }

    builder.append(value);
}

private static String safe(
    String value) {

    return value == null ? "" : value;
}

private static String readPassword() {

    Console console = System.console();

```

```

        if (console != null) {

            char[] chars =
                console.readPassword("Password: ");

            return new String(chars);
        }

        System.out.print("Password: ");

        Scanner scanner =
            new Scanner(System.in);

        return scanner.nextLine();
    }
}

```

5. Compile and run the extractor

```

mkdir bin

javac -cp "lib/*" -d bin src\BOConnectionTest.java src\BOScheduleInventory.java

java -cp "bin;lib/*" BOPROD01:6400 Administrator secEnterprise output\
bo_schedule_inventory.csv

```

6. Expected output

The CSV contains the following initial fields:

```

SI_ID
CUID
Object_Type
Object_Name
Folder_Path
Owner
Publication_Source_Documents
Source_Document_Folders
Publication_Recipients
Destination_Type
Destination_Properties
Schedule_Properties

```

The first extractor run is diagnostic. The raw destination and schedule property columns help identify exactly how BO 4.2 SP5 Patch 3 represents the destinations configured in your system.

7. Target production columns

After inspecting a few masked sample rows, the generic destination properties can be promoted into explicit fields such as:

```

Destination_Type
FTP_Server
FTP_Port
FTP_User
FTP_Path
FTP_File_Name

Email_To
Email_CC
Email_BCC
Email_Subject

FileSystem_Path
BI_Inbox_Recipients

Schedule_Frequency
Next_Run
Last_Run
Output_Format

```

8. Safety and testing notes

Read-only behavior: the programs shown here log in, query repository objects, inspect scheduling/publication metadata, write a local CSV, and log off. They do not call commit/save/delete/schedule operations.

Passwords: the CMS password is prompted at runtime. The extractor also intentionally avoids exporting destination properties that appear to contain PASSWORD.

First test: run BOConnectionTest before the extractor. If compilation fails, retain the complete javac error. If startup fails with NoClassDefFoundError, retain the missing class name. If the test succeeds, run the extractor and inspect two or three masked sample rows.

Compatibility note: BO 4.2 SP5 Patch 3 is an older SDK. A small method/signature difference may appear in the property enumeration portion. If so, the compiler error can be used to adapt that section specifically to the JARs installed in your environment.

9. Relevant SAP SDK concepts

The implementation relies on the BI Platform Java SDK's Enterprise Session and InfoStore APIs. Publication objects expose schedulable source documents and principals, while scheduling information exposes destinations.

Destination property bags can be inspected to identify the destination-specific configuration. FTP destinations also have typed SDK options for server, port, username and destination file information.

For the final production extractor, typed destination interfaces are preferable to raw property IDs once the exact SP5 Patch 3 structures used in your environment are confirmed.